

平时作业二

2312331 王一诺

一、上述语句中 `jbe .L1` 为何这里是 `jbe` 指令？

原 C 代码为：

```
int get_cont(int *p1, int *p2)
{
    if (p1 > p2)
        return *p2;
    else
        return *p1;
}
```

函数参数在栈帧中的位置为：

```
M[R[ebp] + 8] = p1
M[R[ebp] + 12] = p2
```

对应汇编为：

```
movl 8(%ebp), %eax    # eax = p1
movl 12(%ebp), %edx   # edx = p2
cmpl %edx, %eax       # 比较 eax 和 edx, 即比较 p1 和 p2
jbe .L1               # 如果 p1 <= p2, 跳转到 .L1
movl (%edx), %eax     # p1 > p2 时, 返回 *p2
jmp .L2
.L1:
movl (%eax), %eax     # p1 <= p2 时, 返回 *p1
.L2:
```

这里使用 `jbe`，核心原因是：`p1` 和 `p2` 是指针，指针中保存的是内存地址，地址比较通常按无符号数比较；而 `jbe` 正是无符号意义下的“小于等于”跳转指令。

1. `cmpl %edx, %eax` 实际比较的是什么？

x86 AT&T 语法中，操作数顺序是：

`cmpl` 源操作数，目的操作数

`cmpl %edx, %eax` 会计算：

`%eax - %edx`

但不保存结果，只根据减法结果设置标志位。因此这里比较的是：

`p1 - p2`

也就是判断 `p1` 与 `p2` 的地址大小关系。

2. `jbe` 的含义是什么？

`jbe` 是 jump if below or equal，表示“无符号小于等于时跳转”。

它判断的条件是：

```
CF = 1 或 ZF = 1
```

其中：

- `ZF = 1` 表示两个数相等。
- `CF = 1` 表示无符号减法发生借位，即目的操作数小于源操作数。

对于：

```
cmp1 %edx, %eax
jbe .L1
```

含义就是：

```
如果 %eax <= %edx, 则跳转到 .L1
```

也就是：

```
如果 p1 <= p2, 则跳转到 .L1
```

`.L1` 中执行：

```
mov1 (%eax), %eax
```

因为 `%eax = p1`，所以返回 `*p1`。这正好对应 C 代码中的 `else` 分支。

如果 `p1 > p2`，则 `jbe .L1` 不跳转，继续执行：

```
mov1 (%edx), %eax
```

因为 `%edx = p2`，所以返回 `*p2`。这正好对应 C 代码中的 `if` 分支。

因此，`jbe .L1` 的作用是把 `p1 <= p2` 的情况跳到 `else` 分支。

3. 如果多想一点，为什么不用 `jle`？

`jle` 是 signed less or equal，即有符号意义下的“小于等于”。它会把操作数当成有符号整数解释，主要依据 `SF`、`OF`、`ZF` 等标志判断。

但指针是地址，地址本质上不是有符号整数。若把地址当成有符号数比较，最高位为 1 的地址会被解释成负数，从而可能产生不符合地址大小关系的结果。

例如在 32 位系统中：

```
0x80000000
```

如果按无符号数看，它大于：

```
0x00001000
```

但如果按有符号 32 位整数看，`0x80000000` 会被解释成负数，反而小于 `0x00001000`。这显然不适合作为地址大小比较。

所以，指针比较通常使用无符号条件跳转。`jbe` 对应无符号 `<=`，正好适合这里的 `p1 <= p2` 分支判断。

需要补充的是，在标准 C 语言中，只有指向同一个数组对象内部元素，或指向该数组末尾后一位的指针，才可以进行有定义的大小比较。若 `p1` 和 `p2` 指向完全无关的对象，直接比较大小在标准 C 中是未定义行为。但从机器层面看，编译器生成代码时会把指针值当作地址数值处理，而地址比较通常采用无符号比较。

二、`/lib/ld-linux.so.2` 中 `.2` 的含义

`/lib/ld-linux.so.2` 是 Linux 系统中的动态链接器，也叫动态加载器或 ELF 程序解释器。它的主要作用是在程序运行时完成动态链接相关工作，例如：

- 装载程序依赖的共享库。
- 解析共享库路径。
- 完成符号重定位。
- 把控制权交给用户程序的入口。

对于动态链接的 ELF 可执行文件，文件内部通常有一个 `PT_INTERP` 段，用来指定运行该程序时应该由哪个动态链接器启动。例如 32 位 x86 glibc 程序中常见的解释器路径就是：

```
/lib/ld-linux.so.2
```

其中：

```
ld-linux.so
```

表示 Linux 下的动态链接器共享对象。

后面的：

```
.2
```

表示这个共享对象的主版本号或 ABI 版本号。它是动态链接器文件名和 SONAME 的一部分，用来区分不同 ABI 兼容版本的动态链接器。

这里的 `.2` 并不是指：

- Linux 内核版本 2。
- 文件的第二份副本。
- 该程序运行了两次。

它更准确地说是共享对象版本号。对 32 位 x86 的 glibc 环境而言，`ld-linux.so.2` 表示 glibc 2 系列使用的 ELF 动态链接器接口。程序在链接时会把这个路径写入 ELF 文件，运行时内核读取该路径，然后先加载 `/lib/ld-linux.so.2`，再由它继续加载程序依赖的动态库。

可以把它和普通共享库版本号类比：

```
libc.so.6
libm.so.6
libpthread.so.0
ld-linux.so.2
```

这些后缀数字一般表示 ABI 主版本。只要主版本相同，通常表示二进制接口兼容；如果 ABI 发生不兼容变化，就可能需要更换主版本号。

因此，`/lib/ld-linux.so.2` 中 `.2` 的含义是：动态链接器 `ld-linux.so` 的主版本号或 ABI 版本号，用于标识与该可执行程序兼容的运行时动态链接器。

总结

第一题中，`jbe .L1` 是因为 `cmpl %edx, %eax` 比较的是 `p1 - p2`，而 `jbe` 表示无符号意义下的 `p1 <= p2`，正好跳转到返回 `*p1` 的 `else` 分支。指针保存的是地址，地址比较应采用无符号比较，因此这里使用 `jbe` 而不是有符号比较指令 `jle`。

第二题中，`/lib/ld-linux.so.2` 是动态链接 ELF 程序使用的运行时动态链接器，`.2` 是它的共享对象主版本号或 ABI 版本号，表示程序需要与该版本 ABI 兼容的动态链接器来启动和装载。